Verification guide

Rank-six categorification through multiplicity nine

1 October 2026

1 Scope and authoritative records

The package classifies categorifiability of the 25421 rank-six fusion rings of multiplicity at most nine in the published *Fusion Ring Census*. The current ledger has 25362 general exclusions, 56 unitary-positive rings, one spherical-positive ring admitting no pseudo-unitary categorification, and 2 unresolved rings. Positive based rings, rather than the number of constructions, are counted: the 57 positive rings have 64 constructor records.

The authoritative files are `data/census.json`, `data/decisions.json` and `data/classification_counts.json`. The CSV ledger and paper tables are derived from them. The model catalogue records the source, categorical structure and complete basis permutation for each model. The file `data/unresolved_tensors.json` contains all complete remaining tensors. General, pivotal, spherical, pseudo-unitary and unitary existence are separate fields.

The census is pinned to *Fusion Ring Census*, revision

`2a340871242879665fa7ba2b01fa15b8bdeaada3`.

Its rank-six exact-multiplicity counts are $39, 154, 384, 872, 1582, 2768, 3814, 6999, 8809$. The compressed tensor file `data/rank6_tables.txt.gz` has SHA-256

`0e7552e17620452f237e94751c40cd93b0757a466168629db8cc1bf0b9451d00`.

The source and both compressed and uncompressed hashes are recorded in `provenance/repository_source.json`. Import checks fix the input correspondence; completeness of the counting input is supplied by the cited census. The classification does not rerun census enumeration.

2 Checking the release

Use the Python dependencies in `requirements.txt`; C++17 is needed for finite-search helpers and `pdflatex` for rebuilding the documents. From the package root run:

```
python -m pip install -r requirements.txt
python check_manifest.py
python verify.py --jobs 2 --out verification/current/recheck.jsonl
python -O verify.py --jobs 2 --out verification/current/recheck_O.jsonl
python software/current_regressions.py
python -O software/conformal_models.py --verify
python -O software/replay_m9.py --jobs 2
python -O software/replay_m9_structure.py
python -O software/replay_m9_completion.py
python software/release_consistency.py
```

Check the manifest before writing new reports. A new report changes file bytes. The normal and optimized Python checks ensure that correctness does not depend on removable assertions. A subprocess failure has a nonzero return code and must not be reported as a passed check.

The top-level verifier checks the full accepted negative catalogue and positive constructors. The optional `--proofs-only` mode omits positive reconstruction and is therefore not a complete release check. For an independent replay of the accepted continuation certificates use

```
python -O software/continuation_controls.py --jobs 2 \
  --out verification/current/continuation_replay.json
```

This incremental replay does not replace the full-ledger verification. The new multiplicity-nine campaign is recorded separately under `verification/m9/`. Its optimized replay checks every newly allocated general exclusion. The structure-only replay checks the separately registered claims for specified categorical structures; it cannot establish general nonexistence. The additional-criterion campaign has its own source-bound input, attempt ledger and exact replay under `verification/m9/advanced/`. The accepted certificate files retain their frozen tensors, source bindings and input manifests. Their historical filenames are evidence identifiers, not additional current classifications.

A negative checker reconstructs the necessary number fields, finite branch domains, signed quotients, separating vectors and exact inequalities from the full tensor. Numerical optimizers may propose certificates during discovery; their entry points are disabled during negative-certificate checking. An incomplete search, timeout or numerical infeasibility flag cannot establish an exclusion. A successful necessary system cannot establish existence.

These are exact computer-assisted checks, not a proof-assistant formalization. The cited categorical theorems and the proofs of their application remain mathematical inputs. Checksums establish integrity, not the mathematical conclusion.

The multiplicity-five completion. The exclusion of $R^{(6)}_{5,1204}$ first reconstructs its complete 171-profile domain. A forced central product removes 72 profiles; integral cyclic-rotation multiplicities remove the other 99 using 112 certified integer bounds and 99 final exact separating rays. The new proof and solver-disabled replay are under `research/exclusions/R6_5_1204/`:

```
python -O research/exclusions/R6_5_1204/verify_exclusion.py
```

The inherited finite-profile certificate remains part of the proof.

3 Completion at multiplicity nine

Every one of the 8 809 rings of exact multiplicity nine is generally excluded. The eleven final exclusions combine signed dimension-field and different-ideal restrictions, diagonal associativity and trace saturation, complete Frobenius–Schur indicator systems, and a Galois spin-orbit bound. The proof does not assume the pivotality conjecture.

The certificate ledger is `certificates/m9_completion.jsonl`. Portable proof bundles are in `research/exclusions/m9_completion/`; each contains its complete input tensors, mathematical argument and exact finite replay. Run

```
python software/replay_m9_completion.py
python -O software/replay_m9_completion.py
```

The source-bound receipt is recorded in `verification/m9/completion/`. The checker regenerates the required finite domains, verifies every branch and rejects corrupted certificates. All correctness checks remain active with `-O`. The already published counting census and the OEIS terms are unchanged.

4 The three conformal constructions

The positive registry `data/conformal_positive_models.json` adds $R^{(6)}_{4,22}$, $R^{(6)}_{4,91}$ and $R^{(6)}_{4,629}$. Their models are the neutral components of the module categories for $(B_2)_7 \subset (D_7)_1$, $(C_3)_4 \subset (B_{10})_1$ and $(B_2)_{12} \subset (E_8)_1$, respectively. The source files, exact certificates and independent mathematical audit are organized under `research/structural_models/`.

Each conformal embedding supplies an actual unitary algebra and module category. The parity grading gives a rank-six neutral component. Published action graphs or exact free-induction Gram matrices determine a small set of possible fusion tensors. Local modules, duality, formal-codegree induction and the known center's dimensions or indicators distinguish the required tensor. Every resulting tensor is compared in all 216 entries with the census representative. The proofs do not infer a category from dimensions, a modular invariant, or matching modular data.

The canonical command

```
python -O software/conformal_models.py --verify
```

checks the source bindings and exact tensor identifications. The per-model directories `quartic/`, `c3_model/` and `r629/` contain the reconstruction programs and mathematical proofs, including the deeper graph, Weyl-character and indicator checks. These constructions prove existence. They do not classify all associators on the three rings or identify every archived modular-data matrix in a prescribed labelling.

5 The remaining problems

The canonical index `research/frontier/` describes the two remaining general existence questions, their necessary reductions and exact replay commands. No multiplicity-nine case remains open.

The two inherited residuals through multiplicity eight retain the following reductions. For $R^{(6)}_{4,155}$, the generalized near-group fusion rules give a uniform sphericality and Galois-pseudo-unitarity proof by one norm argument. It replaces the former signed-table search for this ring. Fermion fixed-point traces and odd Gauss sums leave one necessary induction profile, of center rank 52, whose twist spectrum is forced and has exponent 24. They replace the former rank-44 integer-bound certificate. The short exact replay is `research/frontier/R6_4_155/verify_center_cuts.py`. Its pointed center also rules out every braided representative of the Morita class. The new exact reconstruction determines every entry of the rank-26 centralizer's modular datum. Its Verlinde coefficients are nonnegative integers, with maximum 9. It forces ten self-dual objects in that factor and twenty in the full center. All stated finite ordinary and generalized indicator checks and all ordered rank-26 Vafa equations pass. No categorical construction follows.

For $R^{(6)}_{8,1660}$, the Ising subcategory makes its complement a regular bimodule. Its relative center has rank eighteen and rules $B[t]/(t^2 - At - L)$, where B is the Ising-center ring. Formal-codegree divisibility and dimension-field constraints give 58 labelled choices. A central second-indicator obstruction and a Morita-invariant invertible count leave one based ring, up to exchanging the Ising factors. The proof and exact replay are in `research/frontier/R6_8_1660/relative_center/`. The retained full-center certificates force exponent dividing 48. Fermion orbit constraints and a weighted second-indicator congruence reduce their 1764 labelled profiles to 1727. The tensor involution and homogeneous-packet Vafa equations then leave 1670; the exact replays are in `research/frontier/R6_8_1660/fermion_profiles/` and `research/frontier/R6_8_1660/spin_profiles/`. A subsequent signed Galois–fermion four-divisibility condition removes raw profile 133 in low frame zero, leaving 1669 profiles in branches

163, 753, 753. Independently, every indecomposable module category has six simples; full-center saturation, integral Morita Gram reconstruction and a weighted second-indicator obstruction leave three possible dual based rings, of ranks six, six and eight.

The new exact calculations and independent audits are under `research/frontier/new_research/`. From the package root, run

```
python research/frontier/new_research/replay.py
```

The current summaries are in `data/frontier_research_progress.json`. The older `data/frontier_summary.json` and source-bound per-ring readmes retain their exact historical-stage bytes.

None of these necessary alternatives is a categorical model. A claimed future exclusion must cover the entire necessary domain; a claimed model must have a categorical construction and complete tensor identification.

6 Rebuilding the presentation

After checking the retained evidence, regenerate the ledger exports, model catalogue, tables and PDFs with

```
python software/release.py
python software/model_catalog.py
python software/build_paper_tables.py
python software/build_readme.py
python build_papers.py
```

Run discovery or regeneration in a working copy, because these commands write files. The build publishes PDFs only after successful compilation. All counts must agree with the decision ledger. The supplement retains accepted proofs, needed dependencies and current residual evidence; obsolete searches for the three conformally realized rings are omitted.